



MODEL
CONTEXT
PROTOCOL



LEARN



BUILD



INTEGRATE

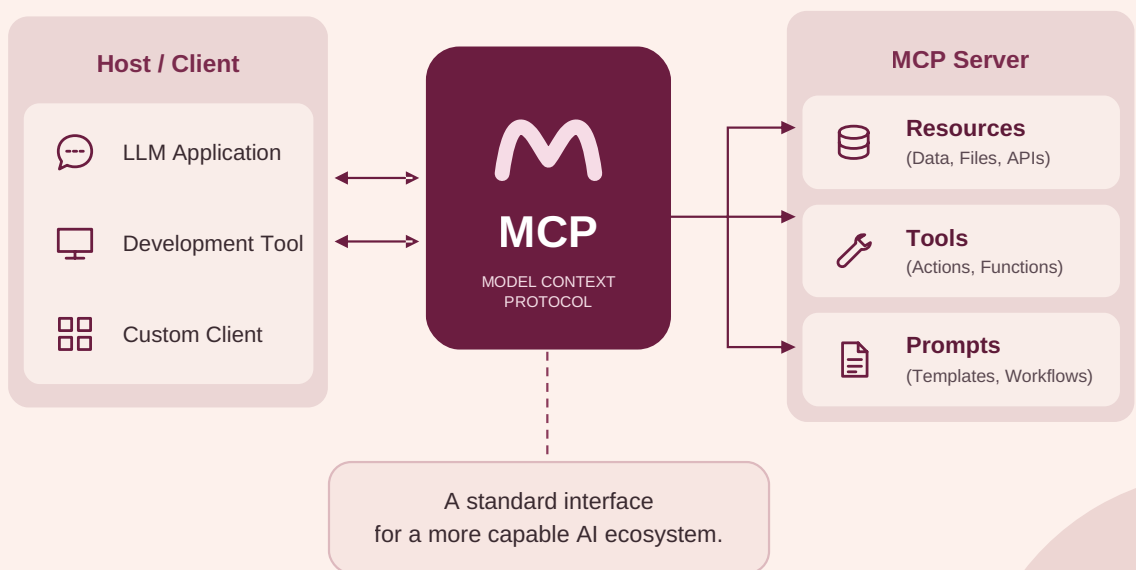


EXTEND

CHAPTER 02

The Why

From the rise of LLM chatbots to the fragmentation and $N \times M$ integration problem that made a shared protocol necessary.



Premium Revision Notes

CONCEPTS

Clear Explanations

IMPLEMENTATION

Practical Code

DIAGRAMS

Visual Learning

INTERVIEW PREP

Key Questions

Chapter 02 — The Why

The Trailer showed MCP already at work — a single AI application reaching across several tools to get a real task done. This chapter goes back to the beginning and answers the question that matters more than any protocol detail: *why did the industry need MCP at all?* The answer is a short history — from the arrival of large language models, through the fragmentation and integration problems that followed, to the specific gap MCP was built to close.

This is a story of escalating solutions, each one solving the previous problem while quietly creating a new one. Understanding that chain is what makes MCP's design decisions in later chapters feel inevitable rather than arbitrary.

1. The Arrival of LLMs

Large language model chatbots reached the public in late 2022 and were adopted faster than almost any software before them — crossing a million users within days and a hundred million within two months. What made this different from prior software wasn't scale alone; it was a new kind of capability. For the first time, people could interact with a machine in natural language and get a genuinely useful response back, rather than performing a narrow, predefined action and waiting for a fixed result.

REAL WORLD INSIGHT

Human-computer interaction had been transactional for decades: press a switch, get light; type a formula, get a number. LLM chatbots introduced something closer to a conversation — the machine could be reasoned with, not just operated.

Three waves of adoption

That first wave of interest played out in three distinct stages, each expanding what the technology was used for.

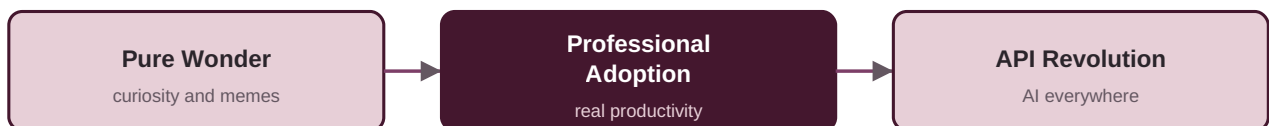


Figure 2.1 — Adoption moved from novelty, to real productivity, to AI embedded inside existing software via APIs.

Wave	What happened
Pure Wonder	People asked playful, curiosity-driven questions and shared the responses on social media. Fun, but not yet useful work.
Professional Adoption	People began pasting real work into the chatbot — contracts to summarize, code to debug, curricula to plan — and measured productivity gains directly.
API Revolution	The underlying models were exposed via API, so existing software (office suites, IDEs, browsers) could embed chat capability directly, rather than requiring a separate chatbot tab.

🔄 Quick Revision

- LLM chatbots offered a fundamentally new interaction style: natural-language conversation instead of fixed actions.
- Adoption moved through three waves: curiosity, real productivity, then API-driven integration into other software.
- The API wave is what let AI capability spread into tools people already used daily.

2. The Problem of Fragmentation

Once model APIs were public, most of the software people already used — note-taking apps, team chat, IDEs — added their own AI features. This sounds like unambiguous progress, but it created a new problem: each of these AI features lived in its own isolated world. The AI inside a notes app had no idea what the AI inside a team-chat app was discussing, and a coding assistant had no visibility into a separate meeting-notes tool. Getting even a small task done meant manually gathering information from one AI-enabled tool and carrying it, by hand, into another.

COMMON MISTAKE

It's tempting to assume that adding AI to more software automatically compounds its usefulness. In practice, each isolated AI feature is only as useful as the context a person manually feeds it — more disconnected AI surfaces can mean more manual copying, not less.

Vision vs. reality

The original hope was a single, unified AI agent that understood a person's entire body of work end-to-end and could help with any part of it on demand. What actually shipped instead was a collection of separate, capable-but-isolated AI tools — genuinely useful individually, but requiring a person to manually bridge the gaps between them. Closing that gap meant solving a more fundamental problem first: **context**.

3. What Is Context?

In the simplest terms, *context* is everything an AI model can see at the moment it generates a response. More formally: context refers to the information an LLM uses to produce its output — conversation history, documents, code, or any other material made available to it.

In a simple chat, context is just the conversation so far — if you ask "how difficult is this to learn?" right after discussing quantum physics, the model resolves "this" from the preceding messages. But context gets far harder to assemble once the task is a real piece of professional work.

A worked example: shipping a feature

Consider a software developer assigned to add two-factor authentication to a product. Getting that task done well requires pulling context from several unrelated places: the ticket description, the existing codebase, the current database schema, the company's security guidelines, and any relevant team discussion.

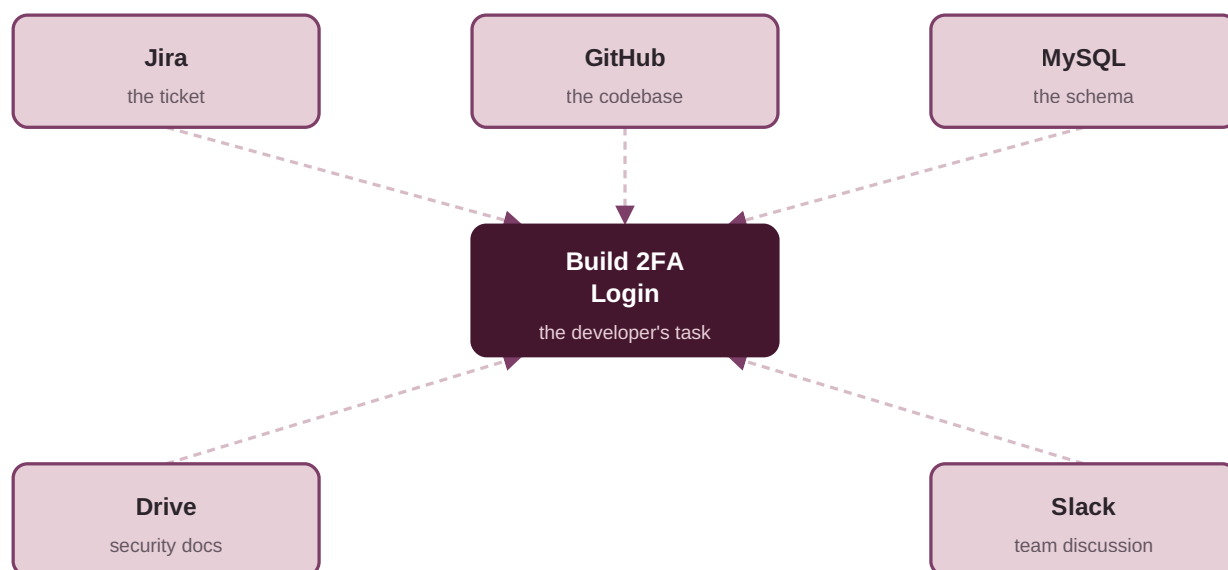


Figure 2.2 — A single real task draws context from several unrelated systems — the context itself is scattered, not centralized.

Without any integration between the AI assistant and these systems, the developer becomes what is effectively a **manual context-assembly service**: copy the ticket into the chat, copy relevant code files, copy the schema, copy the security document, copy the Slack thread — and only then ask the actual question. Every additional source is more manual work, and the process breaks down completely once a codebase or document set is too large to copy and paste at all.

IMPORTANT

The bottleneck was never the model's reasoning ability. It was the manual labor of collecting and transcribing context from every system the task touched — a process that does not scale with task size or with the number of tools involved.

🔄 Quick Revision

- Context = everything the model can see when generating a response.
- Simple chats keep context in conversation history alone; real work scatters it across many systems.
- Without integration, a person must manually copy every piece of context into the chat before asking a question — this is the core scaling problem MCP eventually addresses.

4. The First Fix: Function Calling

Model providers addressed part of this problem in mid-2023 with **function calling**: a way to give a model a list of callable functions, each with a description, so the model could decide when a user's request matched one of them and request that it be invoked with the right arguments.



Figure 2.3 — Function calling lets the model recognize a task and request a specific registered function, instead of only producing text.

This was a small idea with a large consequence: for the first time, a model could do more than converse — it could trigger real actions and fetch real data on request, rather than requiring a person to paste that data in manually.

The rise of tools

Once function calling existed, providers and companies rushed to wire up functions — commonly called **tools** — for the systems people relied on most: weather APIs, internal databases, source-code repositories, web search, and more. Any team could give their AI application a growing library of tools, each one internally responsible for talking to one external system.

Category	Examples of tools built
Enterprise software	CRM integrations, team-chat bots, document/drive connectors, database query tools, source-control integrations
Internal company tools	HR data access, accounting-system integrations, campaign-management tools, infrastructure monitoring
AI-first products	Local file-system access, web browsing and live retrieval, file upload and code execution, computer-use control

BEST PRACTICE

Function calling's real contribution wasn't any single tool — it was proving that an LLM could reliably decide *when* to call external code, which made building an ecosystem of tools around a model worth investing in.

The scattered task, revisited

With tools in place, the same two-factor authentication task looks very different. Rather than manually copying the ticket, code, schema, security document, and chat thread, the developer can simply ask the AI application to fetch each one directly — it already has a tool wired up to the ticketing system, the code host, the database, the document store, and team chat. The context is assembled automatically, and the developer can go straight to asking the real question.

🔄 Quick Revision

- Function calling (mid-2023) let a model request the invocation of a registered external function with specific arguments.
- This enabled **tools** — functions wired up to a specific external system — to be attached to an AI application.
- With enough tools connected, context assembly for a real task can happen automatically instead of by manual copy-paste.

5. The Integration Problem at Scale

Tools solved manual copy-paste, but they introduced a different problem once an organization used more than one AI application. Each tool is, underneath, a custom function written specifically for one AI application's code — its own authentication method, its own way of formatting a request, its own error handling. Wiring up a second AI application to the same external services means writing that integration code again, from scratch, inside a completely separate codebase.

$3 \times 4 = 12$ separate integrations to build and maintain

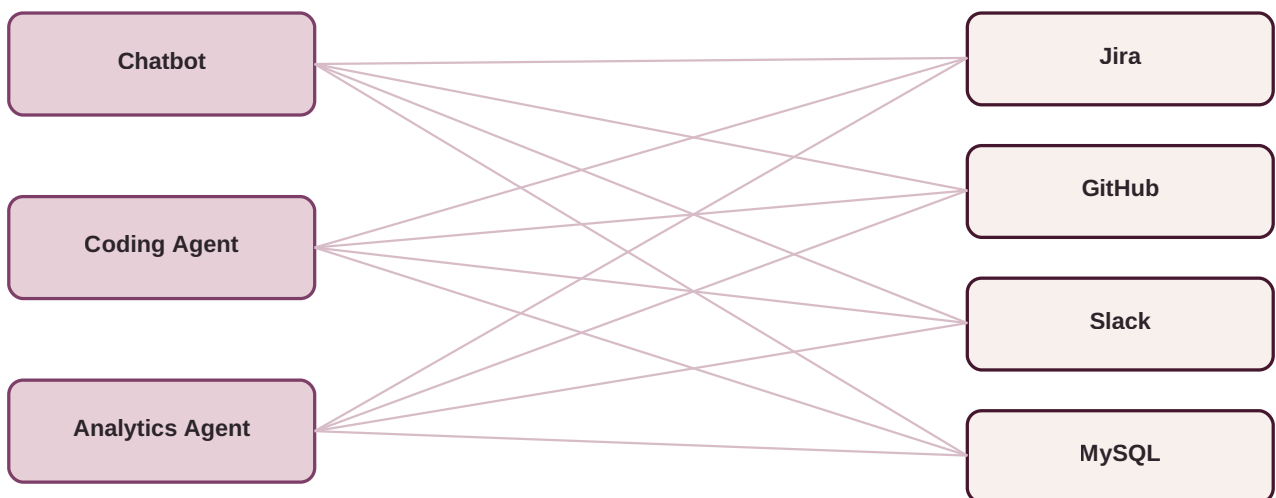


Figure 2.4 — With N AI applications and M external services, every pair needs its own hand-written integration: $N \times M$ integrations to build and keep working.

With three AI applications — say, a general chatbot, a coding agent, and an analytics agent — and a modest twenty external services, that arithmetic already produces sixty separate integrations. Every one of them needs its own authentication, has to track upstream API changes, and is a candidate for a security review.

Cost of the $N \times M$ approach	Why it hurts
Development effort	Every AI application $\times$ every service is a fresh integration to design, write, and test
Maintenance burden	An upstream API change breaks every integration that touched it, across every application
Security exposure	Credentials and access logic are duplicated across many separate codebases instead of one
Cost and time	Fully wiring up one AI application to twenty services can take months and a dedicated team

COMMON MISTAKE

The irony is easy to miss: tools were adopted specifically to reduce manual effort, yet at organizational scale the $N \times M$ integration burden can end up requiring *more* engineering headcount than the problem it was meant to solve.

Standing back, the shape of the problem becomes clear: every AI application was independently inventing its own way to call the exact same external APIs. A service's API doesn't change depending on who's calling it, yet a chatbot, an IDE plugin, and an analytics agent would each ship a separate, incompatible integration to reach it.

Quick Revision

- Each hand-written tool integration is tied to one specific AI application's codebase.
- N AI applications and M services naturally produce $N \times M$ integrations to build and maintain.
- The real problem: every AI application re-invents its own way of calling the same external service.

6. The Shift: Let the Service Build It Once

The fix follows directly from naming the problem correctly. If every AI application is separately reinventing how to call the same service, the solution is for that **service itself** to build one standard integration that any AI application can use — shifting the integration work from $N \times M$ hand-written copies down to one per service, built and maintained by the people who know that service best.

For this to work, every AI application and every service need to agree on a common language to speak to each other in — a shared protocol, rather than each service inventing its own private interface. That shared protocol is the **Model Context Protocol**.

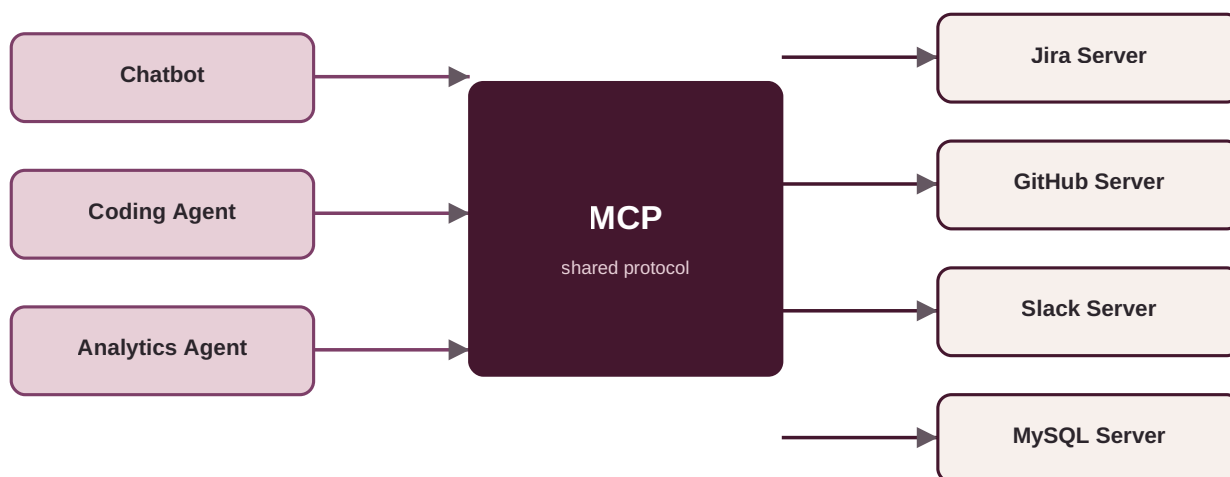


Figure 2.5 — With a shared protocol in the middle, each AI application and each service only need one integration — with MCP itself.

REAL WORLD INSIGHT

This is the same pattern that made earlier technology standards succeed: USB let any peripheral work with any computer without a custom driver from every manufacturer for every machine; MCP aims to do the same for AI applications and the services they need to reach.

Why this is expected to keep growing

The moment a handful of well-known AI applications publicly committed to supporting MCP, popular services came under pressure to build an official MCP integration of their own — because whichever services connect this way become instantly usable by every MCP-compatible AI application, with zero custom code on either side. Each new MCP server makes the ecosystem more valuable to every existing MCP client, and each new MCP-compatible client makes every existing server more valuable in turn — a self-reinforcing network effect that is why MCP is expected to become a default expectation rather than an optional extra.

Quick Revision

- The fix: shift integration work from every AI application to the service itself, built once.
- This requires a shared protocol both sides agree to speak — the Model Context Protocol.
- Network effects (more servers make clients more valuable, and vice versa) are what's driving rapid MCP adoption.

Key Takeaways

- LLM chatbots introduced a genuinely new interaction style, adopted in three waves: curiosity, real productivity, and API-driven integration into existing software.
- Widespread AI adoption created a **fragmentation** problem — isolated AI features with no shared awareness of each other, forcing manual context assembly for any real task.

- **Context** is everything a model can see when responding; for real work it is scattered across many disconnected systems, not held in one place.
- **Function calling** (mid-2023) let models invoke registered external functions, giving rise to **tools** that could fetch context automatically.
- At scale, hand-written tools produce an **$N \times M$ integration problem** — expensive to build, harder to maintain, and a security liability.
- The fix is to move integration work to the service itself, built once against a **shared protocol** — the Model Context Protocol — rather than once per AI application.

Revision Sheet

Adoption

Curiosity → real productivity → API-driven integration into existing software.

Fragmentation

Many isolated AI features, none aware of the others; context assembly stayed manual.

Context

Everything the model can see when responding — scattered across systems for real tasks.

Function calling

Lets a model invoke a registered external function with the right arguments.

The $N \times M$ problem

N apps $\times$ M services = $N \times M$ hand-written, separately maintained integrations.

The fix

Each service builds one integration, once, against a shared protocol — MCP.

Interview Questions

- Q1.** What made LLM chatbots a fundamentally different class of software compared to what came before?
- Q2.** Describe the three waves of chatbot adoption and what changed between them.
- Q3.** What is the "problem of fragmentation," and why did adding AI to more software make it worse rather than better?
- Q4.** Give a formal definition of "context" in the LLM sense, and explain why it's harder to assemble for real professional tasks than for a simple chat.
- Q5.** What is function calling, and what specific capability did it add to LLMs?
- Q6.** Explain the $N \times M$ integration problem in your own words, with an example.
- Q7.** Why does moving integration responsibility to the service itself solve the $N \times M$ problem?
- Q8.** What network effect is driving the growth of the MCP ecosystem?

Further Reading

- OpenAI's function calling announcement and documentation — the mechanism that made tool use possible.
- Model Context Protocol overview — the shared protocol introduced to solve the $N \times M$ integration problem, covered in depth starting next chapter.
- Case studies of early MCP server adoption by major service providers.